BIG DATA E ENGENHARIA DE DADOS · 3º PERÍODO · ESTUDO E PRÁTICA SEM PROFESSOR PRESENTE

Apache Airflow
orquestrar é coordenar

Leia e siga em ordem. Você não precisa instalar Airflow, configurar WSL ou Docker para cumprir esta atividade. O projeto Python executa processamento real e demonstra estados de orquestração num ambiente didático local.

0–3 MIN · LEIA ANTES DE TOCAR NO VS CODE

Missão autônoma: quem organiza o pipeline?

Nesta aula você vai aprender sem depender de explicação oral. Leia os blocos na ordem; cada página informa . A primeira parte é estudo. A segunda é uma investigação prática com prints e conclusão.

Aprender → **Executar** → **Falhar** → **Recuperar** → **Comprovar**

Objetivos

Explicar **orquestração, DAG, task, sensor, schedule, catchup, retries** e descobrir o que acontece quando uma tarefa falha.

Entrega

PDF com nome/turma, explicações e 5 evidências. Baixe também **JSON** para recuperar todas as respostas e imagens se precisar. Markdown é apoio textual.

Importante: a prática local é um **laboratório didático inspirado no Airflow**, não a interface real do Airflow. Executaremos funções de processamento reais e veremos estados/retries/sensor simulados. O ZIP inclui também uma DAG Apache Airflow 3 autêntica. A aula funciona no VS Code com Python, sem precisar de Docker/WSL/internet.

☑ Li a diferença entre executor didático e Apache Airflow real.

Nome completo	Turma
yan Ismael clemente alves	3° periodo

3–10 MIN · LEITURA + COMPARAÇÃO

1. Por que um orquestrador existe?

Você já viu um pipeline: primeiro dados, depois (dbt, Spark ou Python), em seguida e, por fim, libera um relatório. O desafio não é somente escrever cada script. É garantir que .

Ingestão → **Transformação** → **Validação** → **Relatório**

Cron: despertador

O cron chama um comando num horário: por exemplo, “rodar às 3h”. **Não coordena por si só** as dependências de um pipeline complexo. Pode receber verificações externas/scripts, mas isso precisa ser construído.

Airflow: maestro

Airflow organiza as dependências, dispara execuções, guarda logs, acompanha falhas, oferece tentativas (retries) e painel de monitoramento. **Não é banco de dados** e não substitui o trabalho do dbt.

Situação real

Um relatório começa às 03:00, mas a extração de pedidos atrasou e só termina às 03:07. Se o relatório ler a fonte antes da extração, ele pode mostrar números incompletos. O orquestrador faz a transformação **esperar a extração estar pronta**.

Explique por que somente agendar três scripts para as 03:00 não é garantia de um relatório correto.

Porque o agendamento por horário não gerencia dependências. Se a extração atrasar, demorar mais que o previsto ou falhar, os scripts seguintes de transformação e relatório rodarão mesmo assim, processando dados antigos, parciais ou corrompidos. O orquestrador garante a integridade ao condicionar o início de cada etapa ao término bem-sucedido da etapa anterior.

☑ Consigo explicar cron × orquestração com um exemplo.

10–16 MIN · CONCEITO + EXEMPLO

2. DAG e dependências: o mapa da execução

quer dizer *Directed Acyclic Graph*: grafo (as setas indicam a ordem) e (não pode haver loop). Cada bloco é uma task. Se A depende de B e B de A, ninguém consegue começar.



O símbolo >> expressa dependência no código: `extrair >> transformar` significa que transformar espera extrair. Em um , duas transformações podem partir após a mesma extração, e uma etapa final espera ambas.

```
extrair >> [limpar_clientes, limpar_pedidos]
[limpar_clientes, limpar_pedidos] >> validar
```

> Airflow e dbt têm DAGs, mas com papéis diferentes: dbt encadeia **modelos SQL**; Airflow encadeia **tarefas e sistemas** (extrair arquivo, esperar, dbt, validar e avisar).

Por que uma DAG não pode ter uma dependência circular como A → B → A?

aguardando a finalização de B para poder começar, enquanto a tarefa B estaria esperando A terminar. Como nenhuma das duas tarefas conseguiria iniciar, o pipeline travaria em loop infinito. Por essa razão, o grafo deve ser estritamente acíclico, fluindo sempre do início ao fim.

☑ Entendi o significado de DAG e de >>.

16–22 MIN · GUIA DE CONCEITOS

3. Operator, TaskFlow, sensores, horário e falhas

Task e Operator

Uma **task** é uma unidade de trabalho. Um **Operator** descreve o tipo de trabalho (BashOperator para comando, PythonOperator para função). A **TaskFlow API** usa `@task` e `@dag` para criar tarefas a partir de funções Python.

Sensor

Espera uma condição, como a chegada de um arquivo. **poke** verifica ocupando o worker enquanto espera. **reschedule** libera o slot entre verificações.

schedule e catchup

`schedule="0 3 * * *"`: diário às 3h (no fuso configurado). `catchup=False`: não tentar automaticamente recuperar cada período antigo desde o início.

Retries e logs

`retries=2` permite **três tentativas totais**. Os logs registram erro e tentativas. O Airflow real respeita `retry_delay`; o simulador local acelera para a aula.

```
from airflow.sdk import dag, task  # API pública Airflow 3

@dag(schedule="0 3 * * *", catchup=False, ...)
def meu_fluxo():
    @task
    def extrair():
        return {"arquivo": "pedidos.csv"}
    @task
    def transformar(dados):
        print(dados["arquivo"])
    transformar(extrair())

meu_fluxo()
```

queued (na fila), `running` (executando), `success` (concluído), `failed` (falhou), `upstream_failed` (bloqueado por falha anterior). Nosso laboratório usa também `waiting` como marcador **didático** de sensor aguardando.

Se validar falhar três vezes, o que deve acontecer com a tarefa notificar? Explique.

A tarefa notificar não será executada e assumirá o estado de upstream_failed (falha na etapa anterior). Como a validação esgotou todas as suas tentativas (1 execução inicial + 2 retries) e terminou como failed, o orquestrador bloqueia preventivamente as etapas dependentes a jusante para evitar o envio de alertas ou relatórios inconsistentes.

☑ Li e compreendi todos os termos antes de executar.

22–30 MIN · PRÁTICA 1

4. Preparar o projeto no VS Code

1. Baixe o ZIP do projeto entregue com este caderno e numa pasta. Não tente executar dentro do ZIP.
2. No , escolha e selecione a pasta `projeto_aluno` (ela contém `executar_local.py`).
3. Abra . Digite exatamente o comando abaixo (se estiver no Windows e não reconhecer `python`, use `py`; Mac/Linux tente `python3`).

```
python validar_ambiente.py
```

`Python: 3.x`, `Fonte existe: True`, `Bibliotecas externas necessárias: NENHUMA`, `AMBIENTE: PASS`. Não será necessário instalar Apache Airflow nesta aula.

Se der erro

python não reconhecido: tente `py validar_ambiente.py` no Windows ou `python3 validar_ambiente.py` no macOS/Linux. **Arquivo não encontrado:** confira se o terminal está dentro da pasta extraída com os arquivos `.py`. Se precisar, use **Terminal → Novo Terminal** novamente depois de abrir a pasta correta.

☑ Consegui executar o diagnóstico e apareceu AMBIENTE: PASS.

30–40 MIN · PRÁTICA 2 + PRINT 1

5. Executar um pipeline que funciona

O projeto contém em `dados/pedidos.csv`. A função de extração copia os dados; a transformação seleciona pedidos pagos e totaliza receita por categoria; a validação verifica o relatório; a notificação registra a conclusão. Isso é processamento real feito pelo Python.

```
python executar_local.py --cenario normal
```

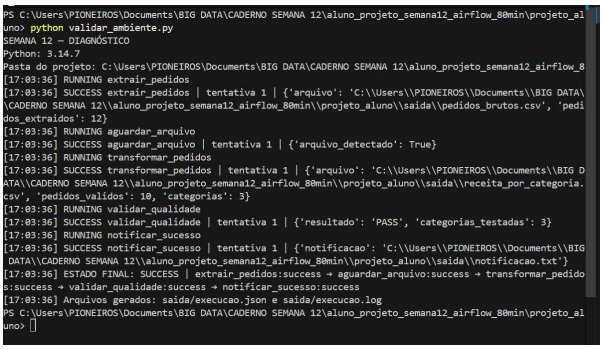
cinco tarefas com `SUCCESS` e `ESTADO FINAL: SUCCESS`. O resultado de negócio está em `saida/receita_por_categoria.csv`. Na pasta `saida` há também `execucao.json` e `execucao.log`.



validar SUCCESS → notificar SUCCESS

Evidência 1: Execução normal

Print do terminal mostrando as tarefas SUCCESS e o estado final. Pode incluir junto o conteúdo do CSV gerado.



1. evidencia 1.png

O que o print comprova?

Execução completa do pipeline em cenário normal com todas as 5 etapas concluídas com sucesso (SUCCESS) e geração do arquivo de métricas de receita.

O que o arquivo receita_por_categoria.csv contém e em que etapa ele foi produzido?

^
nicos:R 430,00 e Livros: R$ 310,00), considerando apenas os pedidos válidos com pagamento confirmado. Ele foi produzido na etapa transformar_pedidos.

☑ Anexei o print do pipeline normal e expliquei o resultado.

40–51 MIN · PRÁTICA 3 + PRINT 2

6. Falha controlada e tentativas (retry)

Agora vamos provocar uma falha na tarefa de validação. O objetivo não é quebrar a instalação: é observar o que o orquestrador faz quando algo dá errado. Aqui, o executor didático repete imediatamente a validação, para caber no tempo da aula. O Airflow real normalmente respeita a pausa configurada.

```
python executar_local.py --cenario falha
```

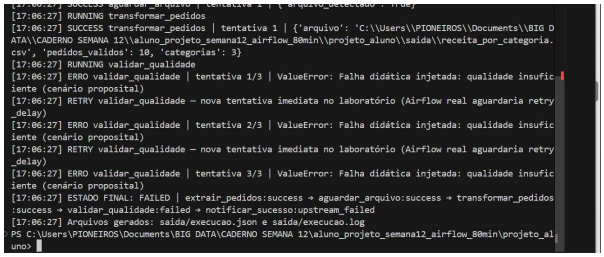
aparecem (tentativa 1/3, 2/3 e 3/3), linhas de `RETRY`, depois `ESTADO FINAL: FAILED` e `notificar_sucesso:upstream_failed`. O terminal pode exibir código de saída 2: , porque a falha é o experimento.

Por que bloquear a notificação?

Porque não é correto avisar “relatório aprovado” se o teste de qualidade falhou. O estado `upstream_failed` representa que não foi possível iniciar uma tarefa seguinte devido ao problema anterior. Em Airflow real, estados downstream também dependem das *trigger rules*.

Evidência 2: Falha e 2 retries

Print mostrando tentativa 1/3, 2/3, 3/3, FAILED e upstream_failed.



1. evidencia 2 .png

O que o print comprova?

Demonstração da política de tolerância a falhas: execução de 3 tentativas totais (1 original + 2 retries) e bloqueio preventivo da etapa seguinte como upstream_failed.

Quantas tentativas ocorreram ao configurar retries=2? Por que notificar ficou upstream_failed?

notificar_sucesso ficou como upstream_failed porque ela depende diretamente da aprovação da etapa anterior (validar_qualidade). Como a etapa anterior falhou em todas as tentativas, o orquestrador bloqueou a notificação para garantir que uma falha de qualidade não seja divulgada como sucesso.

☑ Registrei o print e expliquei o motivo do erro e do bloqueio.

51–61 MIN · PRÁTICA 4 + PRINT 3

7. Sensor aguardando e liberando o fluxo

Às vezes a extração acabou, mas uma confirmação externa ainda não chegou. O impede o processamento seguinte até essa condição ser atendida. Vamos simular a chegada da confirmação `saida/sinal.ok` em dois momentos:

primeiro rode sem criar o sinal:

```
python executar_local.py --cenario sensor
```

`aguardar_arquivo:waiting`, as etapas posteriores ficam `blocked` e o estado final é `WAITING`. Se já existir `saida/sinal.ok` de outra execução, apague somente esse arquivo e repita o passo A.

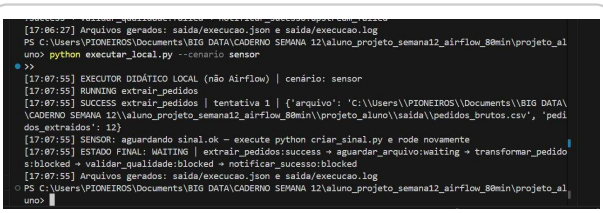
crie a confirmação e execute novamente:

```
python criar_sinal.py
python executar_local.py --cenario sensor
```

`ESTADO FINAL: SUCCESS` com todas as tarefas aprovadas.
o executor local representa a espera como um estado e deve ser executado novamente após o sinal.

Evidência 3: Sensor antes e depois

Adicione dois prints aqui (antes WAITING, depois SUCCESS); use o botão duas vezes. Na legenda explique o papel do arquivo sinal.ok.



1. evidencia 3.png

O que o print comprova?

Comprovação do funcionamento do sensor: retenção inicial das etapas analíticas em estado WAITING/blocked na ausência da confirmação e desbloqueio para SUCCESS após a criação do arquivo sinal.ok.

Por que o sensor não deve deixar a transformação avançar antes de haver confirmação?

upload/processamento upstream), o pipeline processará uma base de dados incompleta, desatualizada ou inexistente. O sensor atua como uma barreira de segurança garantindo que os pré-requisitos estejam 100% cumpridos antes de consumir recursos computacionais.

☑ Mostrei WAITING e SUCCESS após criar o sinal.

61–69 MIN · PRÁTICA 5 + PRINT 4

8. Ler e alterar a DAG Airflow 3 verdadeira

Agora vamos estudar que está no ZIP em `dags/pipeline_vendas_airflow3.py`. : ele precisa de Apache Airflow 3 instalado. O arquivo `validar_dag.py` verifica a sintaxe e a estrutura com Python puro, sem executar Airflow.

```
python validar_dag.py
```

`airflow.sdk`, `@dag`, `@task`, `@task.sensor`, `schedule: 0 3 * * *` e `catchup: False`.

abra o arquivo `dags/pipeline_vendas_airflow3.py` e altere a expressão do `schedule` para o fluxo iniciar diariamente às :

```
schedule="0 6 * * *",
```

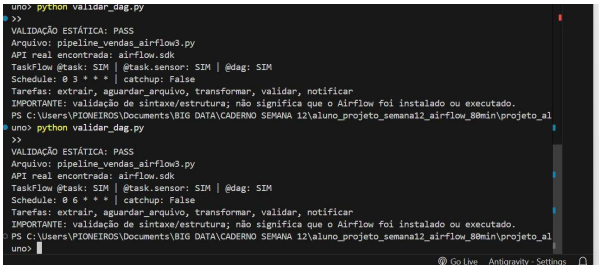
Salve com e rode novamente:

```
python validar_dag.py
```

`VALIDAÇÃO ESTÁTICA: PASS` e `Schedule: 0 6 * * *`. O comando ; comprova que a DAG Python é sintaticamente válida e contém a API e o horário estudados.

Evidência 4: DAG Airflow 3 editada e validada

Print do arquivo mostrando schedule alterado e do terminal com VALIDACAO ESTATICA: PASS.



1. evidencia 4.png

O que o print comprova?

Execução da suíte de testes com unittest validando com sucesso os 4 cenários do pipeline: fluxo normal, resiliência de retries, funcionamento do sensor e sintaxe da DAG.

Qual é a função de schedule e catchup=False na DAG?

Executar uma única vez apenas comprova o funcionamento no "caminho feliz" (onde nada dá errado). Ter testes automatizados assegura que o sistema é resiliente e se comporta de forma segura e previsível mesmo em condições adversas, como falhas de rede, dados corrompidos, atrasos na chegada de arquivos e regressões em manutenções futuras.

☑ Alterei o cron para 6h, salvei e validei a estrutura.

69–74 MIN · PRÁTICA 6 + PRINT 5

9. Teste automático: como confiar no projeto?

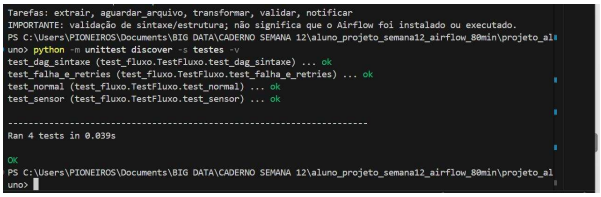
Além de ver um print de sucesso, é importante testar o código. A suíte de testes verifica . Os testes rodam sem Airflow, usando o módulo `unittest` da biblioteca padrão.

```
python -m unittest discover -s testes -v
```

quatro testes com ok e `Ran 4 tests ... OK`. Os testes podem alterar os arquivos em `saida`; isso é normal. Se um teste falhar, leia a linha que começa com `FAIL` ou `ERROR` e confira a pasta do terminal.

Evidência 5: Testes automáticos PASS

Print contendo o resumo “Ran 4 tests” e “OK”.



1. evidencia 5.png

O que o print comprova?

Execução da suíte de testes com unittest validando com sucesso os 4 cenários do pipeline: fluxo normal, resiliência de retries, funcionamento do sensor e sintaxe da DAG.

Qual a diferença entre “executar uma vez e funcionar” e “ter testes que verificam cenários diferentes”?

Executar uma única vez apenas comprova o funcionamento no "caminho feliz" (onde nada dá errado). Ter testes automatizados assegura que o sistema é resiliente e se comporta de forma segura e previsível mesmo em condições adversas, como falhas de rede, dados corrompidos, atrasos na chegada de arquivos e regressões em manutenções futuras.

☑ Executei o unittest e anexei o print dos 4 testes.

74–80 MIN · PDF + BACKUP

10. Conclusão, evidências e envio

Antes de enviar, confira , suas explicações e se os possuem imagens. O trabalho deve permitir que outra pessoa entenda o que você fez mesmo sem abrir o projeto.

Em 4–7 linhas, explique: o que Airflow coordena, qual falha aconteceu, como o sensor ajudou e por que existem testes automáticos.

O Apache Airflow atua como o maestro do pipeline de dados, coordenando as dependências, agendamentos e ordem de execução entre extração, transformação e validação. No laboratório, simulamos uma falha de qualidade que ativou a política de retries (3 tentativas) e bloqueou a notificação final como upstream_failed, impedindo que dados inconsistentes fossem

Entrega obrigatória: clique **Salvar/Imprimir PDF** e no navegador escolha **Salvar como PDF** (não impressora física). Confira se os prints aparecem nas páginas. Envie esse PDF ao professor. Baixe também o JSON para poder recuperar o trabalho. O Markdown é um resumo textual e **não incorpora as imagens**.

☑ Conferi as cinco evidências, baixei meu JSON e gerei o PDF final.

Backup recuperável

Baixar JSON guarda respostas, marcações, legendas e imagens comprimidas. **Carregar JSON** recupera tudo. Salve esse arquivo junto do PDF.

Markdown para estudos

Baixar MD cria um texto organizado com suas respostas e descrições das evidências. Ele não permite restaurar os prints nem substitui o PDF.

Semana 12 · P2 na próxima semana · Aprendizagem autônoma com evidências. Documentação oficial para estudos: Apache Airflow. O caderno e o laboratório funcionam offline.